Nahraďte soubor title-pages.pdf
s Vašemi titulními stránkami
vygenerovanými ze STAGu.

Replace the title-pages.pdf
file with your own title pages
generated from STAG.

Multiplatformní desktopová aplikace pro vizualizaci a analýzu telemetrických dat z testovacích jízd automobilů

ABSTRAKT

Sem vyplňte abstrakt své práce v češtině.

Klíčová slova: DOPLŇTE, SEM, KLÍČOVÁ, SLOVA

Multiplatform desktop application for visualization and analysis of telemetry data from car test drives

ABSTRACT

Insert the abstract of your theses in English here.

Keywords: INSERT, KEYWORDS, HERE

ACKNOWLEDGEMENT

I would like to acknowledge everyone who contributed to the creation of this fine piece of work.

Bc. Petr Boháč

CONTENTS

List of tables 6

List of images 7

List of abbreviations 8

1 Introduction 9

2 Problem analysis 10

2.1 Motivation 10
2.2 Data collection 10
2.2.1 OBD-II-Based Data Acquisition 11
2.2.2 Instrumented Vehicle Data Acquisition 12
2.3 Current workflow 13
2.3.1 Data acquisition 14
2.3.2 Data analysis 14
2.4 Existing Solutions 14
2.4.1 Professional Automotive Engineering Tools 15
2.4.2 Diagnostic and OBD-Based Tools 15
2.4.3 Telemetry and Data Analysis Platforms 16
2.4.4 Conclusion on Existing Solutions 16

3 Building the application 18

3.1 Selecting the programming language 18
3.2 Desktop application frameworks 20
3.3 Development environment 21
3.4 Application architecture 23
3.5 Visualization 23
3.6 Analysis 24

4 Deploying to production 25

4.1 Testing 25
4.2 User documentation 25
4.3 Packaging 25

5 Possibilities for improvement 26

6 Conclusion 27

Bibliography 28

LIST OF TABLES

3.1 Comparison of programming languages for desktop application development 20

LIST OF IMAGES

2.1 A snippet of the predefined tests with various parameters. 10

2.2 ODIS Factory Diagnostic OEM Pro Package used in Škoda Auto. Source: https://diagnoex.com/products/vw-audi-odis-scan-tool 11

2.3 ETAS Hardware for Data Acquisition. Source: https://www.etas.com/ww/en/products-services/data-acquisition-processing-tools 12

2.4 Structure of the Excel spreadsheet after importing the CSV data. 14

LIST OF ABBREVIATIONS

OBD-II	On-Board Diagnostics II
ECU	Electronic Control Unit
ODIS	Offboard Diagnostic Information System
PWM	Pulse-Width Modulation
INCA	Integrated Calibration and Acquisition System
MDF	Measurement Data Format
IDE	Integrated Development Environment

1 INTRODUCTION

In 2025, the Czech automotive giant Škoda Auto produced over 1,043,900 vehicles [1]. At the time of writing, the company is actively manufacturing and selling around 11 different models, including the popular Škoda Octavia, Škoda Fabia, and Škoda Superb. It is pretty reasonable to say that there is a lot of work being done on the research and development side to keep up with demand for new cars and to keep improving existing models.

The process of designing and manufacturing a car is highly iterative. Theory alone can get you relatively far, but at the end of the day, even in a strictly controlled environment, many factors can affect the car's performance, and the only way to really understand how it performs is to test it in real life. Testing is conducted by taking the car on a standardized test drive on a public road, a test track, or in a wind tunnel.[1]. Depending on the test, specific parameters of the drive are standardized, such as speed, drive duration, and, in the case of a wind tunnel, air pressure, temperature, humidity, and the track's incline.

During the drive, a wide array of sensors collects data on various aspects of the car's performance, generating a large amount of telemetry. Tests may be conducted multiple times a day. After a while, the amount of collected data becomes quite large, making it difficult to analyze it all manually. This is where the need for a software solution arises.

The first step in the process will be to analyze the engineers' and technicians' current workflow for data analysis and identify pain points in their process. This will involve conducting interviews and observations to understand how they currently analyze the data, which tools they use, and the challenges they face. Based on this analysis, we will search for existing software solutions that could address these pain points and evaluate their suitability for our specific use case. If we find a suitable solution, we will then customize it to fit our needs.

If no suitable solution is found, requirements for a new software solution will be drawn up, including all the specific features and functionalities users might need for an efficient workflow. An important part of this process will also be to define the software's technical stack and architecture, ensuring it is scalable and maintainable for future developers.

[1]Processes described are specific to the Thermosystem department of Škoda Auto, but the general principles are likely to be similar across the whole company

2 PROBLEM ANALYSIS

As previously mentioned, the development process of a car generates a large amount of telemetry data that needs to be analyzed by engineers and technicians. The current workflow for analyzing this data is inefficient and time-consuming, which motivates the need for a software solution that can streamline the process.

2.1 MOTIVATION

Let's consider the following hypothetical scenario: blah blah tohle nefunguje tak udelame test, ten nam nameri tyhle senzory a znich pozname co zpusobuje ten problem, mrd there u go proto delame testy

2.2 DATA COLLECTION

Because different conditions reveal different aspects of a car's performance, a single standardized test is not sufficient. Instead, cars must be tested across a spectrum of environments. These environments include, but are not limited to, public roads, test tracks, and wind tunnels. By adapting each test to a specific area of investigation, engineers can gain deeper insights into how a car behaves under particular conditions, which is crucial for optimizing its design and performance. A small selection of tests used in production is illustrated in Figure 2. 1, taken from the internal configuration system, also developed by the author.

Název	Identifikátor	Prostředí	$T_{W_HWK_ein}$ (°C)	$T_{L_AUSSEN_s}$ (°C)	$T_{L_AUSSEN_e}$ (°C)	ETD_s (°C)	ETD_e (°C)	$T_{o\text{-}mot}$ (°C)	ATD (°C)	Rychlost (km/h)	Náklon (%)
6% stoupání 45°C	*6% stoupání 45°C*	Tunel	108,00	45,00	45,00	63,00	63,00	150,00	-	-	6,00
8% stoupání 45°C	*8% stoupání 45°C*	Tunel	108,00	45,00	45,00	63,00	63,00	150,00	-	-	8,00
FT mit Anhänger	*FTmit*	Tunel	108,00	24,00	24,00	84,00	84,00	150,00	-	70,00	9,00
FT mit Anhänger REAL	*Ftmit*	Silnice	117,00	30,00	24,00	87,00	93,00	150,00	-	70,00	-
GG mit Anhänger	*GGm*	Tunel	108,00	30,00	18,00	78,00	90,00	150,00	50,00	30,00	12,00
GG mit Anhänger REAL	*GGm*	Silnice	117,00	30,00	18,00	87,00	99,00	150,00	50,00	30,00	-
GG ohne Anhänger	*GGoA*	Tunel	108,00	30,00	18,00	78,00	90,00	150,00	40,00	55,00	12,00

Figure 2. 1: A snippet of the predefined tests with various parameters.

In general, vehicle telemetry data can be collected in two primary ways: through the integrated On-Board Diagnostics II (OBD-II) port [2], or by equipping the vehicle with a large number of additional sensors while simultaneously aggregating data from the car's internal systems.

2.2.1 OBD-II-BASED DATA ACQUISITION

The former method is significantly more straightforward and is commonly used for basic diagnostics and performance monitoring. The setup typically involves connecting a data-logging device, most often a laptop computer, to the vehicle's OBD-II port using a standard interface cable [2]. The computer then runs software capable of communicating with the vehicle's onboard systems, allowing it to read and record various parameters from the car's Electronic Control Unit (ECU) [3].

Any software capable of reading data from the aforementioned port can be used for this purpose, and many options are available, both open-source and commercial. At Škoda Auto, part of the Volkswagen Group, the software commonly used for this task is Offboard Diagnostic Information System (ODIS), a Volkswagen Group diagnostic system (see Figure 2. 2). It is licensed to authorized users and provides a comprehensive set of diagnostic functions for vehicle development and service use [4].

Figure 2. 2: ODIS Factory Diagnostic OEM Pro Package used in Škoda Auto.
Source: https://diagnoex.com/products/vw-audi-odis-scan-tool

The output is typically exported in CSV format, which can be easily imported into spreadsheet software or other data analysis tools for further processing. This method is trivial to set up and can provide a wide range of information about the vehicle's operation, including engine speed, coolant temperature, throttle position, and other parameters. However, due to limitations in available signals and sampling rates, it may not capture all the data required for more detailed or high-fidelity analysis.

2.2.2 INSTRUMENTED VEHICLE DATA ACQUISITION

The second method of data collection involves equipping the vehicle with a large number of additional sensors and aggregating data from the vehicle's internal systems using dedicated supporting hardware (similar to Figure 2. 3). Compared to the first approach, this method is significantly more complex and costly, and the installation process typically requires approximately two days. However, it enables the collection of far more comprehensive and detailed telemetry data, which is essential for in-depth analysis and optimization of vehicle performance.

Figure 2. 3: ETAS Hardware for Data Acquisition
Source: https://www.etas.com/ww/en/products-services/data-acquisition-processing-tools

For example, the ECU often regulates certain components through feedback control loops rather than providing direct measurements. A typical case is the main radiator fan, whose operation is controlled using Pulse-Width Modulation (PWM) to adjust the cooling power dynamically. As a result, the fan speed does not directly correspond to the engine speed, and the actual fan rotational speed

is not always available as a measured signal. In such cases, the fan speed may need to be estimated indirectly from related variables such as voltage, current, and PWM duty cycle. By installing an additional sensor that directly measures the fan's rotational speed, the system can provide a precise and readily usable measurement, which significantly simplifies subsequent analysis[2].

In this configuration, the logging device is typically a laptop computer running specialized third-party software designed for high-fidelity data acquisition. At Škoda Auto, the software commonly used for this purpose is Integrated Calibration and Acquisition System (INCA), a commercial tool developed by ETAS GmbH. This software provides advanced capabilities for data acquisition, calibration, and analysis, making it well suited for the complex requirements of high-resolution telemetry data collection [5].

Compared to the OBD-II-based method, this approach generates significantly larger datasets, often containing measurements from thousands of sensors at different high sampling rates. A simple text-based format would be impractical for storing such large volumes of data, so a more sophisticated standardized format is used. The most common format for this type of data is the Measurement Data Format (MDF), which is a binary file format designed specifically for storing large volumes of measurement data efficiently. The MDF format allows for the storage of complex datasets with multiple channels, varying sampling rates, and metadata, making it ideal for high-fidelity telemetry data [6]. The majority of technicians and engineers at Škoda Auto working with version 3 of the MDF format, but slow adoption of version 4 is expected in the near future.

Because this method has had so much time to mature, the software and hardware ecosystem around it is well developed, and the process of collecting and analyzing data is relatively streamlined. This leaves the first method slightly neglected, thus motivating the development of a software solution that focuses on improving the workflow for OBD-II-based data acquisition and analysis.

2.3 CURRENT WORKFLOW

It is no secret that large corporations across all industries prefer to use commercial software solutions for their operations, and the automotive industry is no exception. Microsoft is the dominant provider of office software, and many companies rely on it for almost all of their day to day operations from email, meetings, and document editing to data analysis and visualization.

Microsoft Excel is an excellent tool for many tasks and can be more than sufficient for for even complex data analysis. That it why it is so widely used in

[2] A photograph of the instrumented engine bay cannot be published due to confidentiality restrictions. During measurements, vehicles are equipped with numerous additional sensors and data acquisition cables, producing a dense instrumentation layout that differs substantially from the standard vehicle configuration.

the industry, including at Škoda Auto, where it is the primary tool for analyzing telemetry data collected through the OBD-II port. The current workflow is as follows:

2.3.1 DATA ACQUISITION

Data is collected using ODIS and exported in CSV format. The CSV data is then imported into Excel using its built-in import functionality, which allows users to specify data types and delimiters. Once imported, the Excel spreadsheet with a structure similar to that shown in Figure 2. 4 is saved to a shared network drive, making it accessible to other team members for further analysis and collaboration.

	A	B	C	D	E	F	G	H	I
1	#2026-03-03 12:56:41 TMBNH9NY0SF000226 trasa MB-MIM-MB, polojasno, 6st., test sady kanalu								
2	#Time	08/vnější te	01/Rychlos	01/buzení \	01/roleta ch	08/vysokon	08/vysokon	08/vysokon	08/vysokon
3	#Time	08----	01----	01----	01----	08-Timeout	08-stav top	08-Napájec	08-stav top
4	#ms	°C	km/h	%	%	-	-	-	-
5	4980	12	2	25.39	29	v poř.	v poř.	v poř.	v poř.
6	9856	11.5	5	25.39	29	v poř.	v poř.	v poř.	v poř.
7	15123	11	13	0	29	v poř.	v poř.	v poř.	v poř.
8	19879	10.5	14	25.39	29	v poř.	v poř.	v poř.	v poř.
9	24139	10	8	25.39	29	v poř.	v poř.	v poř.	v poř.
10	29220	9.5	12	25.39	29	v poř.	v poř.	v poř.	v poř.
11	34075	9	6	25.39	29	v poř.	v poř.	v poř.	v poř.
12	39060	8.5	7	25.39	29	v poř.	v poř.	v poř.	v poř.

Figure 2. 4: Structure of the Excel spreadsheet after importing the CSV data.

2.3.2 DATA ANALYSIS

Engineers and technicians use Excel's functions, formulas, and pivot tables to analyze the data. This analysis may include calculating summary statistics, creating charts and graphs, and performing basic data manipulation on a single file. When multiple test runs need to be compared, engineers must manually open several spreadsheets and compare the data side by side. This process can become cumbersome and error-prone as the number of files increases, especially on the corporate-issued laptops, which often have limited processing power, memory and a small screen size.

It is this second part of the workflow that is particularly inefficient and time-consuming, which motivates the effort to find a better solution.

2.4 EXISTING SOLUTIONS

Before developing a custom tool, it is necessary to examine existing solutions for vehicle telemetry acquisition and analysis. These solutions range from professional automotive engineering tools used by manufacturers to more general-

purpose diagnostic or telemetry platforms. While many of these tools provide powerful capabilities, they often target different use cases or impose limitations that make them unsuitable for the workflow described in this work.

2.4.1 PROFESSIONAL AUTOMOTIVE ENGINEERING TOOLS

In the automotive industry, several specialized software platforms exist for high-fidelity data acquisition, calibration, and analysis. These tools are primarily designed for original equipment manufacturers (OEMs) and their suppliers.

One widely used solution is CANoe, developed by Vector Informatik. The software is used for the development, analysis, and testing of electronic control units and distributed networks [7].

Another related tool from the same company is CANape, which focuses on measurement and calibration of ECU parameters during runtime. During measurement, engineers can adjust parameters while recording signals, with communication commonly handled via protocols such as XCP [8].

These tools provide extremely powerful capabilities but are typically expensive, require specialized hardware, and are primarily designed for ECU development rather than lightweight analysis of diagnostic telemetry data.

2.4.2 DIAGNOSTIC AND OBD-BASED TOOLS

Another category consists of tools designed for vehicle diagnostics using the OBD-II interface.

A well-known example is VCDS, developed by Ross-Tech. It is a diagnostic system for Volkswagen Group vehicles (VW, Audi, Škoda, and SEAT) [9].

Although such tools allow access to a wide range of vehicle parameters, their primary purpose is vehicle diagnostics rather than large-scale telemetry analysis. Data export and advanced analytical capabilities are typically limited compared to dedicated analysis platforms.

Some modern solutions attempt to extend OBD-based data analysis into broader telemetry applications. For example, platforms such as OBDAssistant provide real-time monitoring with hundreds of live parameters and AI-assisted interpretation of diagnostic information [10]. However, they are primarily targeted at consumer diagnostics and fleet monitoring rather than engineering analysis workflows.

2.4.3 TELEMETRY AND DATA ANALYSIS PLATFORMS

Several solutions exist that focus on telemetry visualization and analysis rather than diagnostics.

For instance, the AutoPi platform provides hardware and software solutions for collecting data from CAN, CAN-FD, and OBD-II networks and transmitting it to cloud infrastructure for fleet monitoring and analytics [11].

Similarly, specialized telemetry analysis portals are used in industrial contexts to store and evaluate machine data over long periods, allowing users to perform queries, generate reports, and define operational thresholds. These systems often integrate data from multiple hardware platforms and provide web-based dashboards for long-term analysis.

Another example is asammdf, an open-source tool and Python library designed for working with measurement data stored in the MDF format. It provides functionality for reading, writing, and processing MDF files and includes a graphical user interface for browsing signals, plotting telemetry data, and exporting measurements to other formats [12]. Because MDF is widely used in automotive testing environments, asammdf has become a popular tool for engineers and researchers working with recorded measurement datasets. In recent years, the tool has also started to see practical adoption within Škoda Auto, where it is gradually being used by engineers as an auxiliary tool for working with measurement data. However, it primarily focuses on processing MDF files rather than simplifying workflows built around CSV-based diagnostic data.

2.4.4 CONCLUSION ON EXISTING SOLUTIONS

Although the tools described above provide powerful capabilities, several limitations remain with respect to the workflow discussed in this work. Many professional engineering solutions require expensive licenses and specialized hardware. Dispite the fact large corporations may have access to such resources, including Škoda Auto, scope of the user base the results of this thesis is intended for is limited to a single department, and the cost of deploying such tools across the entire user base would be prohibitive.

In addition, most existing tools are designed primarily for ECU development, fleet monitoring, or vehicle diagnostics rather than for the post-processing and comparative analysis of telemetry data exported from OBD-II systems. As a result, working with multiple test runs often requires manual processing across several files, typically using external tools such as spreadsheet software.

Furthermore, many professional platforms rely on proprietary data formats or tightly integrated ecosystems, which complicates the use of simple CSV-based datasets commonly produced during diagnostic measurements. These limita-

tions motivate the development of a lightweight analysis tool specifically tailored to OBD-II telemetry data, with the goal of improving the efficiency and usability of the current workflow.

3 BUILDING THE APPLICATION

TThe topic of multiplatform desktop applications is a very deep one, and it is very easy to get caught up in the details of the various frameworks and technologies available for building such applications. That is why we need to establish some facts and the particular requirements of our application before we can start evaluating the options.

Pretty much all the computers used by non-IT professionals at Škoda Auto run Microsoft Windows, so we can safely assume that the target platform for our application is Windows. However, it is still desirable to have the option to run the application on other platforms such as Linux or macOS, which motivates the need for a cross-platform solution.

3.1 SELECTING THE PROGRAMMING LANGUAGE

As for the programming language and its ecosystem, making the right choice is crucial for the success of the project in the long term, as it decreases the risk of running into insurmountable technical issues during development and maintenance. There are realistically two main approaches: native development using platform-specific tools and languages, or using a cross-platform framework that abstracts away platform-specific details and allows us to write code once and run it on multiple platforms.

Native applications, often written in languages such as C++ or C#, tend to have better performance and can take full advantage of platform-specific features. In particular, C++ is widely used for high-performance desktop applications and offers fine-grained control over system resources, which could make the application more efficient and snappier from a user perspective. However, native development typically requires separate codebases for each platform, which can be costly and time-consuming to develop and maintain.

On the other hand, cross-platform frameworks allow us to write a single codebase that can run on multiple platforms, which can significantly reduce development time and costs. One popular approach is using JavaScript-based frameworks such as Electron, which enables developers to build desktop applications using web technologies like HTML, CSS, and JavaScript [13]. While Electron simplifies cross-platform development and has a large ecosystem, it

is often associated with higher memory usage and performance overhead compared to native solutions.

The main technical requirements for this project were rapid prototyping, ease of development, and maintainability for the author and possibly other developers in the future. C# with the .NET framework is a popular choice for Windows development, but its cross-platform capabilities, while improving, may still introduce limitations depending on the use case. Java is another option that is inherently cross-platform and has a large ecosystem, but it may be unnecessarily complex for a relatively simple desktop application.

Python, on the other hand, is a versatile language with a large ecosystem of libraries and frameworks for building desktop applications, and it has good support for cross-platform development. It is also widely used for data analysis and visualization, which makes it particularly suitable for applications that require such functionality [14, 15].

There are a lot of factors that influenced the choice of Python for this project (as summarized in table Table 3. 1), but the most important ones were the author's familiarity with the language and its ecosystem, as well as the fact that it is a great language for rapid prototyping and development. The ease of development and maintainability were also important considerations, as the application is intended to be used by engineers and technicians who may not have extensive programming experience. Python's simplicity and readability make it an excellent choice for this purpose, as it allows developers to quickly understand and modify the codebase as needed.

Table 3. 1: Comparison of programming languages for desktop application development

Language	Pros	Cons
C++	• High performance • Fine-grained control over system resources • Widely used for high-performance desktop applications	• Requires separate codebases for each platform • Longer development time • Steeper learning curve for non-C++ developers
C# with .NET	• Good performance • Strong support for Windows development • Improving cross-platform capabilities	• May still have limitations on non-Windows platforms • Potential overhead from the .NET runtime
Java	• Inherently cross-platform • Large ecosystem • Good performance	• May be unnecessarily complex for simple applications • Higher memory usage compared to native solutions
Python	• Versatile language with a large ecosystem of libraries and frameworks for desktop applications and data analysis • Good support for cross-platform development	• Generally slower than compiled languages like C++ or Java • May require additional tools for packaging and distribution

3.2 DESKTOP APPLICATION FRAMEWORKS

Software frameworks provide a structured way to build applications by offering pre-built components and tools that handle common tasks, such as user interface design, event handling, and data management. For desktop applications, there are several popular frameworks available for Python, including PyQt, Tkinter and Kivy.

Python is quite often used for building tools with simple graphical user interfaces using lightweight frameworks such as Pygame or the aforementioned Tkinter. Frameworks like these are easy to learn and can be sufficient for basic applications, but they may not provide the level of polish and user experience expected in a professional setting. For a more modern and responsive user interface, frameworks like PyQt or Kivy are often preferred. PyQt is a set of Python bindings for the Qt application framework, which is widely used for developing cross-platform applications with native look and feel. Kivy, on the other hand, is an open-source Python library for developing multitouch applications, which can

also be used for desktop applications. Both frameworks have their own strengths and weaknesses, so the choice between them came down to personal preference of the author.

PyQt has a slightly steeper learning curve and can be more complex to set up, but it offers a more traditional desktop application experience with a wide range of widgets and tools for building complex user interfaces. One particularly powerful feature of PyQt is its `QSettings` system, which provides a convenient platform-independant way to store and retrieve application settings across sessions. For example, on Windows, settings are typically stored in the registry, while on Linux they are stored in configuration files. By using QSettings' simple API (as shown in Listing 1), developers can abstract away these platform-specific details and ensure that user preferences and application state are preserved seamlessly across different operating systems.

```
settings = QSettings()

# Set a value
settings.setValue("username", "JohnDoe")

# Get a value
username = settings.value("username", "defaultUser")
```

Listing 1: Example of the QSettings API

The Qt framework (and consequently the PyQt library) has more than 25 years of development and a large community, which means that it is well-documented and has a wide range of resources available for learning and troubleshooting. It has a robust layout system that allows developers to create complex and responsive user interfaces that adapt to different screen sizes and resolutions. Additionally, PyQt provides a wide range of widgets and tools for building modern desktop applications, which will be touched upon in the next section [16]. All of these factors contributed to the decision to use PyQt for this project, as it provides a powerful and flexible framework for building a professional-quality desktop application with a modern user interface.

3.3 DEVELOPMENT ENVIRONMENT

Development environment refers to the tools and software used by developers to write, test, and debug their code. A good development environment can significantly improve productivity and make the development process more efficient. For this project, the development environment consisted of a combination of an Integrated Development Environment (IDE) and various tools for version control, testing, and documentation.

The primary IDE of choice for this project was PyCharm, a popular Python IDE developed by JetBrains. PyCharm offers a wide range of features that enhance the development experience, including intelligent code completion, code navigation, and integrated debugging tools. A heavier IDE like PyCharm was chosen over lighter code editors such as Visual Studio Code or Sublime Text because of its robust support for Python and its powerful features that can help manage a larger codebase more effectively.

For the environment itself, the Nix package manager was used to create a reproducible development environment. Nix allows developers to define their development environment in a declarative way, specifying the exact versions of tools and libraries needed for the project. This ensures that all developers working on the project have a consistent environment, which can help avoid issues related to dependency conflicts or differences in tool versions while keeping everything isolated from the system environment [17]. The specific configuration used for this project is shown in Listing 2, which includes Python 3.11 and the necessary packages for development. Direnv was also used to automatically load the Nix environment when entering the project directory, further streamlining the development workflow [18].

```
{
  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixpkgs-unstable";

  outputs =
    { nixpkgs, ... }:
    let
      system = "aarch64-darwin";
      pkgs = import nixpkgs { inherit system; };
    in
    {
      devShells.${system}.default = pkgs.mkShell {
        packages = with pkgs; [
          python311
          uv
        ];

        shellHook = ''
          if [ -f .venv/bin/activate ]; then
            source .venv/bin/activate
          else
            uv venv && uv sync && source .venv/bin/activate
          fi

          alias designer="pyqt6-tools designer &> /dev/null"
        '';
      };
    };
}
```

Listing 2: Nix development environment configuration

As the Nix configuration above shows, the package manager of choice for managing Python dependencies was uv, a modern Python package manager written in Rust that provides a fast and efficient way to manage Python packages and virtual environments [19]. It was chosen over more traditional tools like pip and virtualenv because of its speed, ease of use, and built-in support for dependency resolution and virtual environment management. The pyproject.toml file used by the project is shown in Listing 3, which specifies the project metadata and dependencies.

```toml
[project]
name = "graph-insights"
version = "1.10.0"
description = "Inspection tool for ODIS-exported Excel tabular data"
readme = "README.md"
requires-python = ">=3.11"
dependencies = [
  ...
]
```

Listing 3: The pyproject.toml file used by the project

Finally, Git was used for version control, more so for the sake of good practice and maintaining a history of changes rather than for collaboration, as the project was developed solely by the author. Git allows developers to track changes to their code, revert to previous versions if needed, and manage different branches of development. When working on a new feature or a large refactor, branching allows developers to isolate their changes from the main codebase until they are ready to be merged, which can help prevent conflicts, maintain a cleaner commit history, and allows the developer to work on multiple features or bug fixes simultaneously without interference [20]. For this project, a simple branching strategy was used, with a main branch for stable code and feature branches for new development. Regular commits were made to the feature branches, and once a feature was complete and tested, it was merged back into the main branch using the rebase strategy to maintain a clean commit history. The source code for the project was put up on a self-hosted GitLab instance, which provided a private repository for the project and allowed for easy access and management of the codebase.

3.4 APPLICATION ARCHITECTURE

3.5 VISUALIZATION

render framework selfbuilt renderer - layout strategy - backend - matplotlib backend

3.6 ANALYSIS

classic statistical methods machine learning methods supervised vs unsupervised - unsupervised needs to be monitored - we do monitor it so its the way to go

ODIS vs INCA (excel vs dat)

4 DEPLOYING TO PRODUCTION

4.1 TESTING

unit tests pytest, coverage user testing - feedback from users, iterative development

4.2 USER DOCUMENTATION

zensical local docs local web server - pros and cons

4.3 PACKAGING

pyinstaller, nuitka, brief comparison package skripty

5 POSSIBILITIES FOR IMPROVEMENT

rendering framework - figure management (we only do single figures rn)

6 CONCLUSION

conclusion type shit

BIBLIOGRAPHY

[1] Škoda Auto 2025: 1,043,900 vehicles delivered to customers, Europe's third best-selling car brand. Online. [Accessed 1 March 2026]. Available from: https://www.skoda-storyboard.com/en/press-releases/skoda-auto-2025-1043900-vehicles-delivered-to-customers-europes-third-best-selling-car-brand/

[2] J1962_201607: Diagnostic Connector. Online. 12 July 2016. [Accessed 16 March 2026]. Available from: https://saemobilus.sae.org/standards/j1962_201607-diagnostic-connector

[3] ISO 15031-5:2015 Road vehicles — Communication between vehicle and external equipment for emissions-related diagnostics — Part 5: Emissions-related diagnostic services. Online. August 2015. [Accessed 16 March 2026]. Available from: https://www.iso.org/standard/66368.html

[4] Offboard Diagnostic Information System Engineering (ODIS Engineering). Online. [Accessed 16 March 2026]. Available from: https://www.dne-elektronik.de/en

[5] INCA software products: Tools for measurement, ECU calibration, and diagnostics. Online. [Accessed 16 March 2026]. Available from: https://www.etas.com/ww/en/products-services/data-acquisition-processing-tools/software-products/inca-software-products/

[6] ASAM MDF (Measurement Data Format). Online. [Accessed 16 March 2026]. Available from: https://www.asam.net/standards/detail/mdf/

[7] CANoe (Vector Informatik GmbH) — ASAM member directory. Online. [Accessed 16 March 2026]. Available from: https://www.asam.net/members/detail/vector-informatik/

[8] CANape (Vector Informatik GmbH) — ASAM product directory. Online. [Accessed 16 March 2026]. Available from: https://www.asam.net/members/product-directory/detail/canape/

[9] VCDS-Lite: Windows-based Diagnostic Software for VW / Audi / Seat / Skoda. Online. [Accessed 16 March 2026]. Available from: https://www.register.ross-tech.com/vcds-lite/index.php

[10] OBDAssistant: AI-powered OBD2 vehicle intelligence platform. Online. [Accessed 16 March 2026]. Available from: https://www.obd2assistant.com/

[11] AutoPi Cloud and automotive data loggers. Online. [Accessed 16 March 2026]. Available from: https://www.autopi.io/

[12] Welcome to asammdf's documentation! — asammdf 8.7.2 documentation. Online. [Accessed 16 March 2026]. Available from: https://asammdf.readthedocs.io/en/latest/

[13] Introduction | Electron. Online. [Accessed 23 March 2026]. Available from: https://electronjs.org/docs/latest

[14] MCKINNEY, Wes. *Python for Data Analysis*. "O'Reilly Media, Inc.", 2013. ISBN 978-1-4493-1979-3.

[15] Matplotlib — Visualization with Python. Online. [Accessed 27 November 2025]. Available from: https://matplotlib.org/

[16] Qt (software). Online. 2026. [Accessed 24 March 2026]. Available from: https://en.wikipedia.org/w/index.php?title=Qt_(software)&oldid=1343148945Page Version ID: 1343148945

[17] How Nix Works | Nix & NixOS. Online. [Accessed 25 March 2026]. Available from: https://nixos.org/guides/how-nix-works/

[18] direnv – unclutter your .profile. Online. [Accessed 25 March 2026]. Available from: https://direnv.net/

[19] uv. Online. [Accessed 25 March 2026]. Available from: https://docs.astral.sh/uv/

[20] Git. Online. [Accessed 25 March 2026]. Available from: https://git-scm.com/about